김동현(202485-010007)

1. 계좌별 거래금액 등을 효율적으로 통계 처리하기 위해, 객체지향 프로그래밍(OOP) 방식을 활용하여 고객 계좌 관리 프로그램을 작성하고자 한다. 다음 사항들을 만족하는 클래스를 작성하시오 (메서드나 속성의 명칭으로 괄호 안의 이름을 사용할 것)

1-1. Account 라는 이름의 클래스를 생성하시오.

인스턴스 생성 시, **이름(owner)**을 입력받아 등록하고, **잔액(balance)**은 기본값 0으로, **거래 내역(transactions)**은 빈 리스트로 초기화하도록 작성하시오.

In [2]:
```
# 계좌 클래스
class Account:
    # 생성자
    def __init__(self, owner):
        self.owner = owner
        self.banlace = 0
        self.transactions = []
```

1-2. 해당 클래스에 대해 다음의 입금/출금 메서드를 작성하시오.

- **입금(deposit)**: `amount` 를 파라미터로 받아서, 받은 값만큼 잔액을 증가시키고, 거래 내역에는 **양의 값**으로 기록하도록 작성하시오.
- **출금(withdraw)**: `amount` 를 파라미터로 받아서, 받은 값만큼 잔액을 감소시키고, 거래 내역에는 **음의 값**으로 기록하도록 작성하시오.

In [5]:
```
# 계좌 클래스
class Account:
    # 생성자
    def __init__(self, owner: str):
        self.owner = owner
        self.balance = 0
        self.transactions = []

    # 입금 메서드
    def deposit(self, amount: float):
        amount = float(amount)
        self.balance += amount
        self.transactions.append(amount)

    # 출금 메서드
    def withdraw(self, amount: float):
        amount = float(amount)
        self.balance -= amount
        self.transactions.append(-amount)
```

1-3. 해당 클래스에 대해 다음의 메서드들을 작성하시오.

- 잔액 조회(get_balance): 현재 잔액을 반환하도록 작성하시오.

- 거래금액 평균(get_average): 전체 거래금액의 평균을 반환하도록 작성하시오 (입금, 출금 모두 절댓값 기준).
- 거래 횟수(get_transaction_count): 총 거래 횟수를 반환하도록 작성하시오.
- 거래금액 최솟값(get_min): 거래 내역 중 가장 작은 값을 반환하도록 작성하시오 (절댓값 기준).
- 거래금액 최댓값(get_max): 거래 내역 중 가장 큰 값을 반환하도록 작성하시오 (절댓값 기준).

클래스를 만든 후, A부터 Z까지 인물에 대한 계좌 정보를 담은 인스턴스들을 생성하는 다음 코드를 실행하시오.

```
for letter in "ABCDEFGHIJKLMNOPQRSTUVWXYZ":
    account = Account(letter)
    globals()[letter] = account
    for i in [1,4,2,5,3,1,9]:
        account.deposit(ord(letter)*i**2)
        account.withdraw(ord(letter)*i)
```

In [7]:

```
# 계좌 클래스
class Account:
    # 생성자
    def __init__(self, owner: str):
        self.owner = owner
        self.balance = 0
        self.transactions = []

    # 입금 메서드
    def deposit(self, amount: float):
        amount = float(amount)
        self.balance += amount
        self.transactions.append(amount)

    # 출금 메서드
    def withdraw(self, amount: float):
        amount = float(amount)
        self.balance -= amount
        self.transactions.append(-amount)

    # 잔액 조회 메서드
    def get_balance(self) -> float:
        return self.balance

    # 거래 횟수 메서드
    def get_transactions_count(self) -> int:
        return len(self.transactions)

    # 거래금액 평균 메서드
    def get_average(self) -> float:
        # 거래가 없는 경우 평균은 0
        if len(self.transactions) == 0:
            return 0.0
        # 거래가 있는 경우 transactions 리스트의 절대값의 합을 거래 횟수로 나누어
        else:
            sum_transactions = sum(abs(t) for t in self.transactions)
            average = sum_transactions / self.get_transactions_count()
            return average
```

```
    # 거래금액 최소값 메서드
    def get_min(self) -> float:
        # 거래가 없는 경우 최소값은 0
        if self.get_transactions_count() == 0:
            return 0.0
        # 거래가 있는 경우 transactions 리스트의 절대값 중 최소값 계산하여 반환
        else:
            min_transaction = min(abs(t) for t in self.transactions)
            return min_transaction

    # 거래금액 최대값 메서드
    def get_max(self) -> float:
        # 거래가 없는 경우 최대값은 0
        if self.get_transactions_count() == 0:
            return 0.0
        # 거래가 있는 경우 transactions 리스트의 절대값 중 최대값 계산하여 반환
        else:
            max_transaction = max(abs(t) for t in self.transactions)
            return max_transaction
```

In [8]:
```
for letter in "ABCDEFGHIJKLMNOPQRSTUVWXYZ":
    account = Account(letter)
    globals()[letter] = account
    for i in [1,4,2,5,3,1,9]:
        account.deposit(ord(letter)*i**2)
        account.withdraw(ord(letter)*i)
```

(1) B 계좌의 거래금액 평균값(절댓값 기준)을 계산하시오.

In [11]:
```
B.get_average()
```

Out[11]: 763.7142857142857

(2) C 계좌에서 5000을 인출한 후, 잔액을 조회하시오.

In [12]:
```
C.withdraw(5000)
C.get_balance()
```

Out[12]: 2504.0

(3) D 계좌의 거래 횟수를 조회하시오.

In [14]:
```
D.get_transactions_count()
```

Out[14]: 14

(4) E 계좌의 거래금액 최솟값(절댓값 기준)을 계산하시오.

In [15]:
```
E.get_min()
```

Out[15]: 69.0

(5) F 계좌의 거래금액 최댓값(절댓값 기준)을 계산하시오.

In [16]:
```
F.get_max()
```

Out[16]: 5670.0

(6) Account 클래스의 속성 중에서 튜플로 만들어도 괜찮은 속성은 무엇인가?

owner 속성은 튜플로 만들 수 있다. 튜플은 값이 변할 수 없는 특성을 가지고 있기 때문이다.
수시로 값이 변하는 balance 나 transactions 은 튜플로 만들 수 없다.

2. 다음 확률분포와 관련된 문제들을 푸시오.

2-1. $N(3, 2^2)$의 0.3-quantile 값을 계산하시오.

In [28]:
```
from scipy import stats

# 정규분포 설정 (평균=3, 표준편차=2)
mu = 3
sigma = 2
normal_dist = stats.norm(loc=mu, scale=sigma)

# 0.3-quantile 계산
quantile = normal_dist.ppf(0.3)
print(f"0.3-quantile: {quantile:.4f}")
```

```
0.3-quantile: 1.9512
```

2-2. $X \sim N(3, 2^2)$이고, X의 pdf가 $f(x)$일 때, $\int_0^{6.92} f(x)\,dx$ 를 구하시오.

In [29]:
```
# 0~6.92 사이의 확률 계산
prob = normal_dist.cdf(6.92) - normal_dist.cdf(0)
print(f"P(0 ≤ X ≤ 6.92): {prob:.4f}")
```

```
P(0 ≤ X ≤ 6.92): 0.9082
```

2-3. $X \sim Poisson(4.5)$이고, X의 pmf가 $f(x)$일 때, $\sum_{x=2}^{7} f(x)$ 를 구하시오.

In [30]:
```
# 포아송 분포 설정 (평균=4.5)
mu = 4.5
poisson_dist = stats.poisson(mu)

# x=2~7 일 확률 계산
prob = poisson_dist.cdf(7) - poisson_dist.cdf(1)
print(f"P(2 ≤ X ≤ 7): {prob:.4f}")
```

```
P(2 ≤ X ≤ 7): 0.8523
```

3. 다음 행렬 A에 대해 특이값 분해(SVD)를 진행하시오.

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \\ 13 & 14 & 15 \end{bmatrix}$$

3-1. 행렬 A의 우측 특이벡터들을 구하시오.

In [32]:
```
import numpy as np

A = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9],
    [10, 11, 12],
    [13, 14, 15]
])

left_SV, SV, right_SV_T = np.linalg.svd(A)
print("Right Singular Vectors (transposed):")
print(right_SV_T)
```

```
Right Singular Vectors (transposed):
[[-0.51927261 -0.57552072 -0.63176883]
 [-0.75079244 -0.04592639  0.65893966]
 [-0.40824829  0.81649658 -0.40824829]]
```

3-2. 행렬 A의 스펙트럼 노름(spectral norm)을 구하시오.

In [33]:
```
spectral_norm = max(SV)
print(f"Spectral Norm: {spectral_norm:.4f}")
```

```
Spectral Norm: 35.1826
```

3-3. 행렬 A는 상위 몇 개의 특이값/특이벡터로 완벽하게 복원될 수 있는가?

행렬 A를 보면 [열1 + 열3] = 2 × [열2] 이며, 독립적인 열이 2개이다.
따라서 행렬A는 상위 2개의 특이값/특이벡터로 완벽하게 복원할 수 있다.